TP integrador — Algoritmos y Estructuras de Datos

Cátedra: 00184 — Algoritmos y Estructuras de Datos
Cuatrimestre: C2 2026
Lenguaje: Python 3
Modalidad: grupos de **2 o 3** personas · programa de **consola (CLI)**
Entregas: 6 incrementales, domingo 23:59, tag de GitHub `entrega-N`

Este trabajo **reemplaza las prácticas y el segundo parcial**. En clase hay mini-ejercicios para fijar cada tema; el hilo conductor del cuatrimestre es **un mismo sistema que crece entrega a entrega**. La defensa oral de noviembre es la instancia que ocupa el lugar del 2do parcial.

---

1. Qué hay que construir

Un gestor de catálogo en consola, a elección del grupo, en **uno** de estos tres temas:

Tema	Producto	Analogía rápida
Pokédex	Atlas de Pokémon, equipo de combate e historial	Pokédex + equipo de 6
Recetario	Libro de recetas, menú semanal y cola de preparación	Recetas con sub-recetas
Biblioteca musical	Catálogo de canciones, playlists y cola de reproducción	Mini reproductor / biblioteca

Los tres temas son **la misma consigna con otra piel**. Mismas estructuras, mismas entregas, misma rúbrica. Cambian los nombres del dominio.

No se pide interfaz gráfica ni web. Solo biblioteca estándar de Python (`struct`, `pathlib`, `argparse` o menú con `input`, `abc`, `datetime`, etc.). Los datasets son archivos de **texto simple (`.txt`)**: un registro por línea, campos separados por coma, primera línea con los nombres de los campos. Si una librería no viene con Python, no se puede usar.

---

2. Mapa del dominio (el mismo TP, tres pieles)

Concepto de la materia	Pokédex	Recetario	Biblioteca musical
Entidad del catálogo	Pokémon	Receta	Canción
Catálogo	Pokédex	Libro de recetas	Biblioteca
Relación **recursiva**	Cadena de evoluciones	Sub-recetas	Versiones (cover, live, remix)

Concepto de la materia	Pokédex	Recetario	Biblioteca musical
Colección lineal principal	Equipo (máx. 6)	Menú de la semana	Playlist
Pila	Historial de acciones (deshacer)	Historial de recetas cocinadas	Historial de reproducción
Cola	Cola de turnos de combate	Cola de preparación	Cola de reproducción
Búsqueda	Nombre, tipo, nro.	Nombre, ingrediente, categoría	Título, artista, género
Ordenamiento	Ataque, velocidad, nro.	Tiempo, dificultad	Año, duración, título
Archivo de texto	`pokedex.txt`	`recetas.txt` (+ ingredientes)	`canciones.txt`
Archivo binario	registros de Pokémon	registros de receta	registros de canción
Excepciones típicas	Equipo lleno, Pokémon no encontrado	Ingrediente/sub-receta faltante	Cola vacía, canción no encontrada

El dataset inicial de cada tema está en `esqueleto/data/` en archivos de texto simple (`.txt`). Pueden ampliarlo, no pueden vaciarlo. Estos archivos se empiezan a leer en E5; hasta esa entrega el catálogo se arma en el código.

---

3. Requisitos transversales (valen para los tres temas)

3.1 CLI

Menú de texto, claro, que no se rompa con input vacío o basura. Operaciones mínimas al final del cuatrimestre:

1. Listar catálogo (usando el **iterador** de la lista enlazada).
2. Ver detalle de un ítem.
3. Buscar (lineal; binaria cuando el catálogo esté ordenado).
4. Ordenar por al menos dos criterios.
5. Operación recursiva del dominio (sección 2).
6. ABM sobre la colección principal (equipo / menú / playlist).
7. Push/pop de la pila (historial / deshacer).
8. Encolar/desencolar la cola.
9. Guardar y cargar catálogo en **texto simple (`.txt`)** y en **binario**.
10. Salir.

3.2 TADs obligatorios (implementación propia)

Hay que implementar, en módulos separados, con encapsulamiento:

- `Nodo`
- `ListaEnlazada` — insertar (al inicio, al final, en orden), eliminar, buscar, tamaño, **iterador** (`__iter__` / `__next__` o `yield`). No alcanza con un `list` de Python disfrazado.
- `Pila` — **sobre** `ListaEnlazada` (no sobre `list`). `apilar`, `desapilar`, `ver_tope`, `esta_vacia`.
- `Cola` — **sobre** `ListaEnlazada`. `encolar`, `desencolar`, `ver_frente`, `esta_vacia`.

La colección principal del dominio (equipo, menú o playlist) **debe** usar `ListaEnlazada`. El historial usa `Pila`. La cola de turnos / preparación / reproducción usa `Cola`.

El catálogo completo puede vivir en la lista enlazada. Si en la entrega 4 arman un índice auxiliar (`dict`) para discutir complejidad, tiene que estar justificado en el informe: qué ganan y qué pagan.

3.3 Recursión

Una función recursiva de verdad, sobre el dataset, con caso base explícito. Ejemplos:

- Pokédex: imprimir la cadena de evoluciones hacia adelante (y, si pueden, pre-evoluciones).
- Recetario: desglosar una receta en ingredientes totales, explotando sub-recetas.
- Música: listar todas las versiones derivadas de una canción (cover, live, remix).

Incluir en el informe **una traza** de una llamada (cómo se apilan los casos).

3.4 Excepciones

Excepciones propias, como mínimo: ítem no encontrado, colección llena o vacía, pila vacía, cola vacía, archivo inválido. El CLI las captura y muestra un mensaje usable. Nada de `except:` pelado.

3.5 Archivos

Dos formatos, los dos de verdad:

1. **Texto simple (`.txt`) secuencial.** Lectura y escritura completa del catálogo (y tablas auxiliares si aplica: evoluciones, ingredientes, versiones). Un registro por línea, separado por coma.
2. **Binario de registros de longitud fija** con `struct`. Header (magia `AYED`, tema, cantidad) + registros. Poder **actualizar un registro por posición** (acceso directo), no solo reescribir el archivo entero.

Las cadenas en binario van en UTF-8, recortadas o rellenadas con `\0` a un ancho fijo documentado en el informe.

3.6 Búsqueda y ordenamiento (implementación propia)

Prohibido usar `list.sort`, `sorted`, `bisect` o equivalentes para cumplir el requisito. Pueden usarlos en un apéndice comparativo.

- Búsqueda lineal.
- Búsqueda binaria sobre el catálogo **ya ordenado**.
- Al menos **dos** algoritmos de ordenamiento vistos en la materia (por ejemplo selección e inserción; Shell o alguno avanzado suma).

3.7 Complejidad

Tabla en el informe: operación → complejidad en tiempo (y espacio si importa) → breve justificación. Medir tiempos con `time.perf_counter` sobre el dataset y, si pueden, sobre una copia agrandada (repetir filas) para mostrar la tendencia. No hace falta un paper: hace falta que se note que midieron.

3.8 Documentación (va creciendo)

Archivo	Desde
`README.md`	Entrega 1
`docs/INFORME.md`	Entrega 1, se completa en 4 y 6
`docs/PROTOCOLO_PRUEBAS.md`	Entrega 2, ejecutado en 3 y 6
`docs/DECLARACION_IA.md`	Entrega 1, actualizado **en cada entrega**

Idioma del repo: el que elijan, **el mismo** en código, mensajes y docs.

---

4. Cómo se entrega

1. Un único repositorio **GitHub** por grupo (público o privado con acceso a la cátedra).
2. Partir del esqueleto publicado por la cátedra (fork o copia, conservando la estructura de carpetas).
3. **Mail obligatorio del grupo** a **los dos** docentes, en el mismo correo (ver §4.1). Sin ese mail el grupo no está inscripto y E1 no se corrige.
4. En el campus: **URL del repo** + tag. El código que se corrige es el del tag, no el de `main` suelto.
5. Tags obligatorios: `entrega-1` … `entrega-6`.
6. Vencimiento: **domingo 23:59** (hora Argentina) de la fecha de cada entrega.

4.1 Mail del grupo (obligatorio)

Enviar **un solo mail, a ambos**:

- Dr. Diego Agustín Ambrossio (profesor titular) — diego.ambrossio@unab.edu.ar
- Lic. Angel Leonardo Bianco (JTP / consultas) — angel.bianco@unab.edu.ar

Asunto: `[AyED C2 2026] Grupo Apellido1-Apellido2[-Apellido3]`

Cuerpo:

- Nombre, mail y usuario de GitHub de cada integrante
- Tema elegido (Pokédex / recetario / biblioteca musical)
- URL del repositorio (si ya está creado)

Plazo: al armar el grupo, **no más tarde que el domingo 30-ago-2026**. Si el 30-ago todavía no hay repo, el mail sale igual con integrantes y tema; la URL tiene que estar sí o sí en E1.

4.2 Preentrega del repo (opcional, solo para probar)

El **domingo 30-ago-2026** (una semana antes de E1) pueden hacer una **preentrega solo del repositorio**: esqueleto copiado, `README` con nombres, repo accesible. Tag sugerido: `preentrega`.

No se corrige código ni suma nota. Sirve para comprobar que la cátedra puede clonar el repo y que el mail llegó. El programa todavía no hace falta que liste el catálogo: eso es E1.

Atraso: hasta el martes 23:59 siguiente, con **−25 %** de esa entrega. Después no se acepta. La entrega 6 **no admite atraso** (las defensas son el 25 y el 27-nov). Una excepción justificada (salud, trámite) se avisa **antes** del vencimiento.

Defensa oral (25 y 27-nov-2026, 15 minutos): reemplaza al 2do parcial. Todos los integrantes hablan. Pueden pedir cualquier función, traza de recursión, invariante de la pila, formato del binario o un caso del protocolo de pruebas. Si alguien no puede explicar el código que "escribió", esa persona no aprueba la defensa aunque el repo esté bien. Quien falte con justificación rinde oral de recuperatorio en la semana del 02-dic.

---

5. Uso de IA

Está **permitida** (ChatGPT, Cursor, Copilot, etc.) con tres condiciones:

1. Completar `docs/DECLARACION_IA.md` en cada entrega (herramienta, fecha, para qué, qué reescribieron).
2. Entender y poder defender **todo** el código del tag.
3. No entregar un repo que el grupo no pueda modificar en vivo.

No declarar IA cuando se usó es suficiente para anular esa entrega.

---

6. Calendario de entregas

Entrega	Vence	Cubre (clase)	Qué tiene que andar
Preentrega (opcional)	**domingo 30-ago-2026**	Semanas 1–2	Solo el repo: esqueleto, README con nombres, mail del grupo a Ambrossio y Bianco. No se puntúa.
E1	**domingo 06-sep-2026**	Semanas 1–3	Repo, README, integrantes, tema elegido, catálogo armado a mano (los `.txt` en `data/` aún no se leen), listar catálogo, funciones básicas, tipos usados con justificación breve (qué es mutable y qué no). CLI mínimo.
E2	**domingo 20-sep-2026**	Semanas 3–5	Módulos. **Recursión del dominio** con traza en el informe. Objetos / TADs del dominio. Protocolo de pruebas (casos escritos; todavía no hace falta haber corrido todos).

Entrega	Vence	Cubre (clase)	Qué tiene que andar
E3	**domingo 04-oct-2026**	Semanas 5–7	Encapsulamiento. `ListaEnlazada` + iterador. Pila y cola sobre esa lista. Colección principal del dominio. Excepciones propias. Menú CLI usable. Protocolo ejecutado (tabla pasa / no pasa).
E4	**domingo 01-nov-2026**	Semanas 10–11	Búsqueda lineal y binaria. Dos ordenamientos propios. Tabla de complejidad + medición de tiempos.
E5	**domingo 15-nov-2026**	Semanas 12–13	Texto simple (`.txt`) secuencial + binario con `struct`. Alta / baja / modificación persistente. Acceso a un registro por posición.
E6	**domingo 22-nov-2026**	Integración	Producto cerrado: las 10 operaciones del menú, informe completo, protocolo de pruebas de regresión, declaración de IA al día. **Defensa oral: 25 y 27-nov-2026.**

La entrega 3 cierra **diez días antes del 1er parcial**. La 6 vence el 22-nov; las defensas son el 25 y el 27-nov (en el lugar del 2do parcial).

---

7. Evaluación

Instancia	Peso
1er parcial	25 %
Entrega 1	5 %
Entrega 2	7 %
Entrega 3	12 %
Entrega 4	9 %
Entrega 5	9 %
Entrega 6	8 %
Defensa oral (reemplaza el 2do parcial)	25 %

- El 1er parcial se aprueba con **60 %**. Recuperatorio en la semana del 02-dic.
- Las seis entregas se aprueban con **30 / 50**.
- La defensa se aprueba con **15 / 25** y con todos los integrantes presentes.
- Cada entrega se corrige con el checklist de `rubrica.md`. Entrega que no corre (el CLI no arranca) se puntúa 0 en “funciona”.
- Nota extra (hasta 2 puntos sobre las entregas, no pasa de 50): usar la lista enlazada también en el catálogo **y** un índice `dict` comparado en el informe; o un tercer algoritmo de ordenamiento medido.

Las autoevaluaciones, la guía de prácticos y el segundo parcial del programa 2023 quedan absorbidos por este TP.

---

8. Qué no hace falta

- Frameworks, GUI, web, base de datos, pandas, numpy.
- Tests automatizados con `pytest` (el protocolo manual es el requisito).
- Gráficos de Pokémon, audio real, ni una receta que se pueda cocinar de verdad: el valor está en las estructuras.
- Reimplementar el universo completo del tema. El dataset de la cátedra alcanza.

---

9. Armado de grupos

- 2 o 3 estudiantes. No hay grupos de 1 salvo excepción explícita de la cátedra.
- Nombres, mails y GitHub en el `README.md` desde E1.
- El grupo **tiene que avisarse por mail a Ambrossio y Bianco** (consigna §4.1). No alcanza con decirlo en clase.
- Si alguien abandona: avisar por el mismo canal. El resto sigue; no se reabre el cupo después de E2 salvo motivo grave.
- El trabajo tiene que poder explicarse por cualquiera. "Eso lo hizo el otro" no es respuesta válida en la defensa.

---

10. Criterio de "entrega sucesiva incremental"

Cada tag **incluye** lo anterior y **suma** lo nuevo. No se entrega un script suelto distinto cada vez. Si en E4 rompen la recursión de E2, E4 pierde los puntos de regresión (ver rúbrica).

El esqueleto ya tiene las carpetas `src/tads`, `src/algoritmos`, `src/persistencia`, `src/dominio`. Completen ahí. No entreguen un único `tp.py` de 800 líneas.